

Árvores

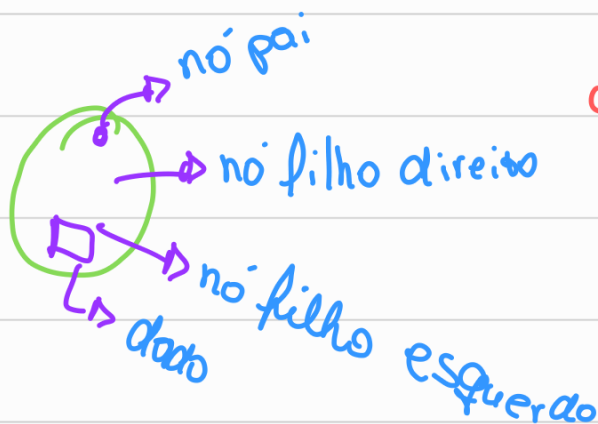
Nó raiz $\rightarrow$ não tem pai

Nó folha $\rightarrow$ não tem filho

a maioria tem grau 2, e são direcionados
(por ponteiros)

em uma árvore bem feita a
distância da raiz a folha é

$$h = \log(\text{vértices})$$



olhando o último número

menores $\leftarrow$ maiores $\rightarrow$

1 $\rightarrow$ raiz (1º elemento)

passa compo.
sendo com todos.



n vai ter cara de
árvore, é mais uma

fila, o intuito é ter o

menor caminho

Árvore bem feita = balanceada

arquivo_código.c

arquivo_cabecalho.h → só tem as assinaturas das
funções, variáveis
globais → 0 & chama a função

a assinatura é o tipo nome(tipo, tipo) int soma(int, int)

Árvores

↳ usa mais números nos nós

Dados Satélites → uma série de informações que vem junto no nó, mas o que mais usa p/ implementar é os dados principais.

só pode dois filhos.

```
Struct No { id
```

```
    dado_satélite (string char[100]
```

```
    nó_pai (struct no* pai
```

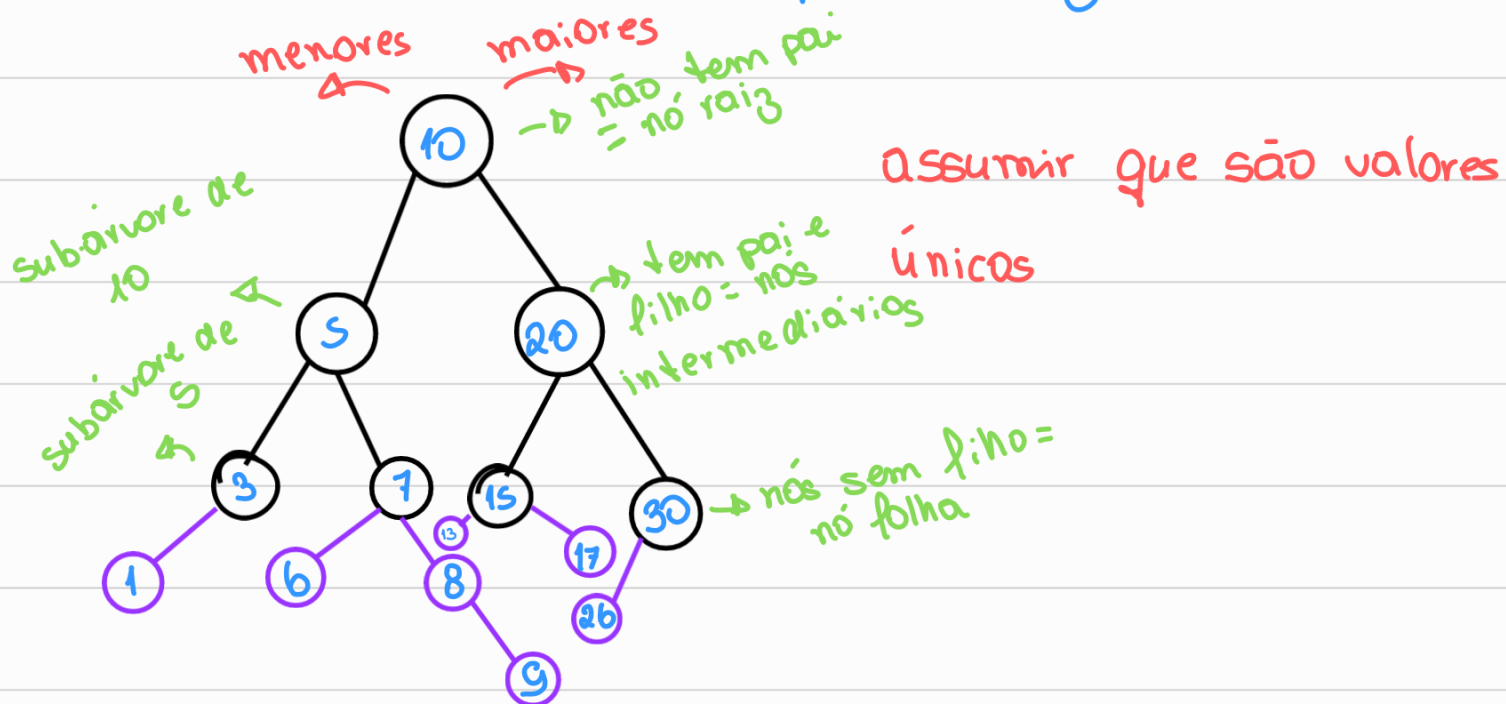
```
    filho_direito (struct no* esq
```

```
    filho_esquerdo (struct no* dir
```

Árvore Binária de Busca

↳ todos os nós à esquerda do nó raiz possuem valores menores que o nó raiz.

↳ todos os nós à direita do nó raiz possuem valores maiores, ou iguais, que o nó raiz.



manipulações nas árvores

inserir o valor 8

↳ ver se é menor ou maior que o nó raiz e ir descendo até um nó sem filhos. Quando achar coloca ali. adicionar = 1, 17, 13, 26, 6, 9

inserção →

pega o id e vai comparando até achar um nó que não aponta p/ ninguém

usa malloc p/ alocar memória p/ novo nó

```
if (nó atual → id < valor) {
```

```
    if (nó atual → esq == NULL) {
```

```
        struct No* novo = (struct No* ) malloc(sizeof(struct No));
```

```
        novo → id = valor
```

```
        novo → esq = NULL
```

```
        novo → dir = NULL
```

```
        novo → pai = nó atual
```

```
        nó atual → esq = novo
```

```
    } else {
```

```
        inserir (nó atual → esq, valor); }
```

achar o menor valor

navegar à esquerda até achar esq == Null

Achar o maior valor

Navegar à direita até achar dir == null

Sucessor de um nó

Navega até o n° e procura o menor valor da subárvore à direita do nó

Antecessor de um nó

o maior valor da sub-árvore à esquerda do nó

Mostrar todos elementos da árvore

In-Order

Mostra todos os elementos em ordem crescente.

1º o que tem à esquerda da raiz (no → esq)

2º a raiz (nó id)

3º o que tem à direita da raiz (no → dir)

```
void in_order(struct NO* atual) { // Post-Order
    if (atual != NULL) {
        in_order(atual->esq);
        printf("%d", atual->id);
        in_order(atual->dir);
    }
}
```

→ ^{melhor} chamada
recursiva ou
do while

Pre-Order

Mostra todos os elementos por ordem de amplitude

1º a raiz (nó id)

2º tudo que tem à esquerda (no → esq)

3º tudo que tem à direita (no → dir)

```

void m_order (struct NO* atual) { → Post-Order
    if (atual != NULL) {
        in_order(atual->esq);
        printf("%d", atual->id);
        in_order(atual->dir);
    }
}

```

} inverte essas duas linhas apenas

Post-Order

Inversão da pre-order

1º todos os nós à esquerda (no->esq)

2º todos os nós à direita (no->dir)

3º nó raiz (no->id)

```

void m_order (struct NO* atual) { → Post-Order
    if (atual != NULL) {
        in_order(atual->esq);
        printf("%d", atual->id);
        in_order(atual->dir);
    }
}

```

} inverte essas duas linhas

Remoção de Nós

⊙ Nó-folha

Acha o nó folha e vai p/ pai dele e apaga o filho

nó->esq = null

```

if( no->id == vala ){
    if( no->esq == NULL && no->dir == NULL ){
        anterior->dir = NULL;
        free(no);
    }
}

```

② Nó intermediário com 1 filho

Vai até o nó que quer apagar, faz o filho dele apontar p/ pai dele e o pai ^{atual} apontar p/ filho dele

```

if( no->id == vala ){
    if( no->esq == NULL && no->dir != NULL ){
        no->dir->pai = anterior;
        anterior->esq = no->dir;
        free(no);
    }
}

```

③ Nó intermediário com 2 filhos

a) Se o sucessor do nó for no->dir

O sucessor não pode ter filhos a esquerda, aí faz os nós à esquerda do que vai excluir ser filho do nó sucessor, depois disso cai no caso 2.

```

if( no->esq != NULL && no->dir != NULL ){
    if( no->dir->id == sucessor(no) ){
        no->dir->esq = no->esq;
        no->esq->pai = no->dir;
        no->dir->pai = no->pai;
        no->pai->esq = no->dir;
        free(no);
    }
}

```

b) Se o sucessor do nó não for nó → dir

Copia no que vc quer apagar todos os valores do sucessor
se ele tiver filhas vai cair no caso 2 ou 3 depois de fazer as
trocas apaga o sucessor.

```
if (no->esq != NULL && no->dir != NULL) {  
    if (no->dir->id != sucessor(no)) {  
        no->sucessor // troca  
        no->id = no->sucessor->id  
        strcpy(no->nome, no->sucessor->nome);  
        remove(no->dir, no->id);  
    }  
}
```